CareNest — Build Prompt for Claude Code

Mission

You are the lead engineer building **CareNest**, a home-healthcare platform for India: residents get doctor consults, medicine delivery, home lab tests, hospital-visit companions and a 24×7 emergency SOS from one app, and operations staff run everything from a web console. Build it production-grade from day one for **Android, iOS and web** from a single TypeScript monorepo. Android and web ship first; iOS development starts within 7 days, so every mobile decision must work on both platforms from the first commit. "CareNest" is a working name that will change; follow the brand rules below so a rename is a config change, not a code change.

Brand and renaming

The final name will be set before the first app store release, so the name, domain and logo must come from one place, and nothing permanent may carry the working name. At the start of Phase 6, ask me for the final name, domain and logo, run `pnpm rebrand`, and work through the rename checklist before any store submission.

- **One source of truth:** a `packages/brand` package exports the name, short name, legal entity name, tagline, domains, support phone and email, WhatsApp display name, SMS sender header and logo files. Every app, email, PDF, push notification and WhatsApp message variable reads from it.
- **No name in code or infrastructure:** never use the brand name in folder, package, table, schema, bucket, environment variable, API route, analytics event or Workflow names, or in Vercel and Supabase project names. Use neutral names such as `resident`, `field`, `console`, `api`.
- **User-facing text:** all copy lives in i18n files and inserts the name with a `{{brand}}` variable. A CI check fails the build if the working name appears anywhere outside `packages/brand`.
- **Permanent identifiers:** the Android application ID and iOS bundle ID cannot change after the first store release. Use neutral, company-based IDs (for example `in.<company-domain>.resident` and `in.<company-domain>.field`), and ask me for the company domain before the first upload. The app name shown on phones and in the stores can change at any time.

- **Links:** use a neutral deep-link scheme, and universal and app links on the domain from `packages/brand`. After a rename, keep the old domain redirecting.
- **Logos and icons:** generate app icons, splash screens and favicons from one source SVG with a script, so a new logo is one file swap.
- **Rename checklist:** keep `docs/rename-checklist.md` listing what needs outside approval on a rename: WhatsApp display name and any templates that mention the name (prefer templates without it), SMS sender header and DLT templates, App Store and Play Store listings and screenshots, Razorpay merchant name, email sending domain, and API key restrictions tied to domains. Add a `pnpm rebrand` script that updates `packages/brand` and regenerates assets.

Product summary

CareNest has five client surfaces and five services; the design files show the resident app, admin console and landing page in three visual themes. **Build the Aurora theme only** (`/design/CareNest-Aurora.html`: deep indigo surfaces, lime accent, Sora headings, DM Sans body, rounded 22 to 26 px corners). Keep every colour, font, radius and spacing value in design tokens and never hard-code them in components, so Midnight, Clinical or a new theme can be added later as a token file only. The theme will likely change, so:

- **Semantic tokens only:** components use role names (`surface`, `text-muted`, `accent`, `danger`, `service-consult`), never raw colours or font names. A lint rule fails the build on hex colours or font families outside `packages/tokens`.
- **Runtime theme provider:** both mobile apps and both web apps read the active theme from one provider, so switching or adding a theme needs no component changes. Support a light and a dark variant per theme from the start.
- **Everything themed:** charts, map styles, heatmap colour ramps, status colours, illustrations, splash screens and email templates read from tokens too.
- **Keep meaning stable:** status colours (danger, warning, success) and service colours keep the same roles across themes, and every theme must pass WCAG AA contrast, checked in CI.

Customers: gated communities and apartments (sold to associations), individual families, senior-living homes and employers. Never hard-code "community" as the only customer type; a household can exist without one.

Surface	Who uses it	Platform
Resident app	Residents and their families	Android, iOS
Field app	Ambulance crew, phlebotomists, visit companions, pharmacy riders	Android, iOS
Admin console	CareNest operations, community desk staff	Web
Partner portal	Doctors, pharmacies, labs, network hospitals (ER handover)	Web (same app as admin, role-based)
Landing site	Prospects, communities signing up	Web

Services:

1. **Doctor consults:** video, audio or in-clinic, booked by specialty and slot, with an e-prescription at the end.
2. **Medicine delivery:** orders from an e-prescription or an uploaded photo, fulfilled by enrolled local pharmacies, checked against the resident's allergies.
3. **Lab tests at home:** sample collection slots, a phlebotomist visit, reports delivered into the app with out-of-range values flagged.
4. **Visit companion:** a verified companion escorts the resident to a hospital visit and back, with live trip updates for family.
5. **Emergency SOS:** one tap alerts the nearest ambulances and the community desk, shows the ambulance live on a map, and sends the resident's medical history to the receiving network hospital before arrival.

Also in scope from the mockups: a personal health record (vitals with trends, conditions, allergies, medicines and adherence, devices, vaccinations, insurance), notifications, bookings, consent controls, a usage heatmap of active users by locality and hour, and a resident heatmap from city view down to a villa or flat site plan.

Tech stack (decided)

Use one TypeScript codebase end to end and deploy everything web-side on **Vercel in the Mumbai region (`bom1`)**: Next.js for the frontend, a Hono API on Vercel Functions for the backend, Vercel Workflows for durable timers, and Supabase in Mumbai (`ap-south-1`) for Postgres, PostGIS, auth, storage and real-time. The two mobile apps are built with Expo

and ship through the app stores, not Vercel. Use the latest stable version of each library when you scaffold, and check its current docs before writing integration code; do not rely on remembered APIs.

Frontend: Next.js (landing site and admin console) on Vercel; React Native with Expo (resident and field apps) on Android and iOS.

Backend: a Hono API (TypeScript) running as Vercel Functions in `bom1`, with Vercel Workflows for SOS escalation and other timers, Supabase for data, auth, files and real-time, and Upstash Redis for live positions and locks.

Layer	Choice	Why this one
Mobile apps	React Native + Expo (development builds, Expo Router, EAS Build and Submit)	One codebase for Android and iOS, so iOS can start in week one at no extra cost. Use development builds, never Expo Go, because calling and SOS need native modules. Built by EAS and shipped through the stores.
Web frontend	Next.js (App Router) with Tailwind CSS and shadcn/ui, two Vercel projects: `web` (landing, SEO) and `console` (admin and partner portal)	Native to Vercel, server rendering for the landing page, and a fast dashboard framework for the console.
API	Hono (TypeScript) deployed as Vercel Functions on Node.js with Fluid compute, region `bom1`; OpenAPI via `@hono/zod-openapi`; Zod schemas shared in a `contracts` package	Hono is small and starts fast on serverless, unlike NestJS. Typed clients are generated from the OpenAPI spec for mobile and web.
Durable timers and jobs	Vercel Workflows (Workflow SDK: `'use workflow'`, `'use step'`, `sleep`, hooks), pinned to `bom1`; Vercel Cron for nightly rollups	SOS escalation (30 s and 90 s), order SLAs and reminders must survive crashes and redeploys. Workflows give durable sleeps and retries without running a worker server.

Layer	Choice	Why this one
Database	Supabase Postgres in Mumbai (`ap-south-1`) with the PostGIS extension, Drizzle ORM, row-level security	PostGIS answers "nearest free ambulance" and powers both heatmaps. Mumbai keeps health data in India and next to the `bom1` functions.
Real-time	Supabase Realtime (broadcast and Postgres changes) for SOS state, live ambulance positions, live queues and console alarms	A managed real-time service with mobile and web clients. Vercel Functions can hold WebSockets, but they close at the function's maximum duration, so SOS should not depend on them.
Live state and locks	Upstash Redis from the Vercel Marketplace (closest region to Mumbai): GEO sets for crew positions, atomic locks for "first crew to accept wins", rate limits	Sub-second lookups during dispatch and abuse protection on OTP and SOS endpoints.
Auth	Supabase Auth with phone OTP, delivered through Supabase's SMS hook as a WhatsApp authentication template first and MSG91 SMS as the fallback; staff sign in with email and TOTP two-factor; roles in JWT claims	Phone OTP is how Indian users expect to sign in. Row-level security enforces who sees which household.
Files	Supabase Storage (Mumbai), private buckets, short-lived signed URLs	Prescriptions, lab reports, uploaded photos. Never public.
Native call UI	`react-native-callkeep` (CallKit on iOS, ConnectionService on Android) + VoIP push (PushKit on iOS, high-priority FCM data messages on Android)	Incoming consult and crew calls must ring like a phone call, even when the app is killed. Wire through Expo config plugins.

Layer	Choice	Why this one
Video and voice calls	LiveKit Cloud (WebRTC); the API on Vercel only issues room tokens	Call media flows between devices and LiveKit, so nothing heavy runs on Vercel. Good React Native and web SDKs, handles poor networks.
WhatsApp and SMS messaging	WhatsApp Business Platform (Meta Cloud API, direct) for alerts, updates, reminders and OTP; MSG91 SMS as the fallback when a WhatsApp message is not delivered	Most users in India read WhatsApp sooner than SMS. SMS stays as a DLT-compliant fallback so no alert depends on one channel.
Push notifications	Firebase Cloud Messaging (Android) and APNs (iOS) via `expo-notifications`; iOS Critical Alerts entitlement for SOS	SOS alerts to crew must break through silent mode. Apply to Apple for Critical Alerts in week one.
Maps and location	Google Maps Platform: `react-native-maps`, Routes API for ETAs, Maps JavaScript API with a deck.gl heatmap layer on the web; `expo-location` + `expo-task-manager` for background location	Best Indian address and road coverage.
Payments	Razorpay (UPI, cards, subscriptions) with webhooks to the API	Consult fees, medicine and lab orders, and per-home plans.
Hosting and deploys	Vercel Pro team (Hobby is for non-commercial use): one Vercel project each for `web`, `console` and `api` from the same repo, all pinned to `bom1`; preview deployments per pull request; Supabase branching for preview databases	One platform for every web deploy, with previews that the team can click through.

Layer	Choice	Why this one
Monorepo	pnpm workspaces + Turborepo with Vercel Remote Cache	Shared types, design tokens, API client and lint config across all apps; fast builds on Vercel.
Quality and monitoring	Vitest, Playwright (web), Maestro (mobile end-to-end), Sentry on every app, Vercel Observability for functions and workflows	SOS and calling must be tested on real devices and monitored in production.

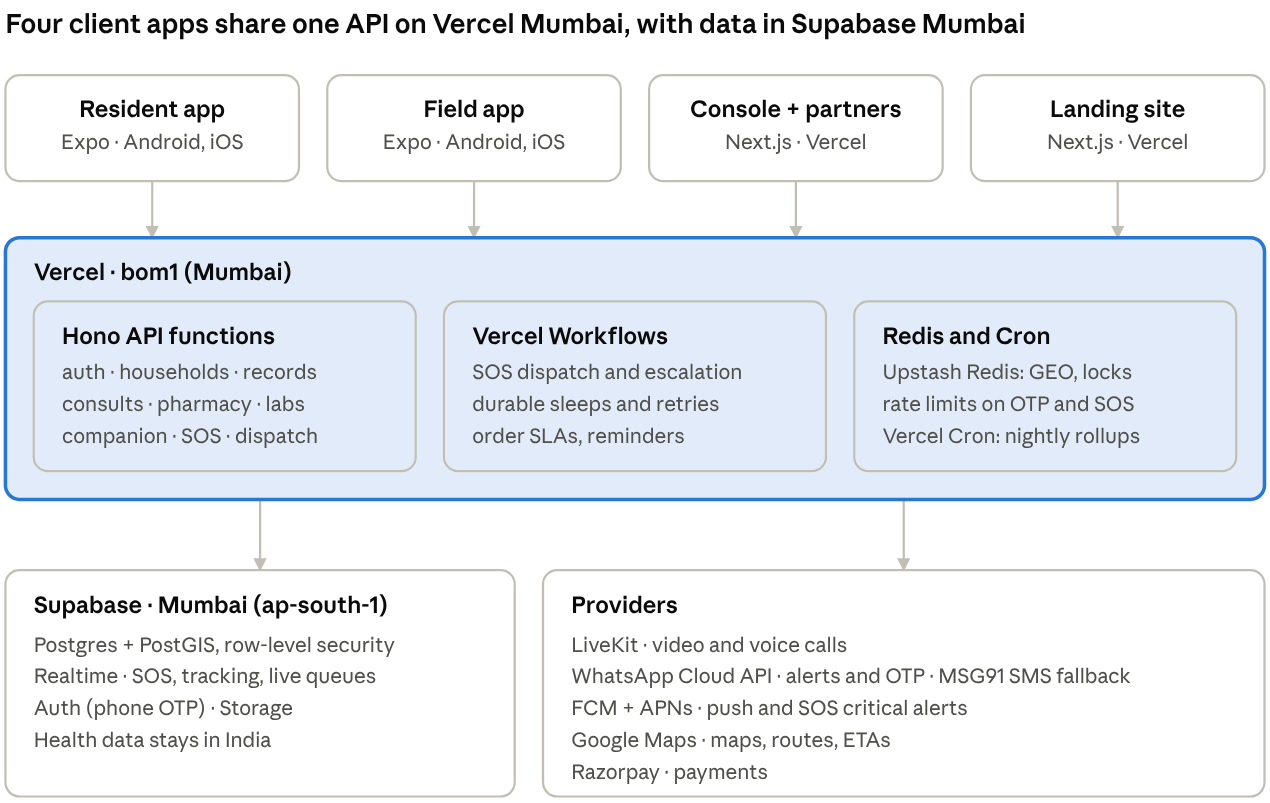
Repository and architecture

Set up this monorepo layout before writing features; every app consumes shared packages rather than copying code.

```
<repo>/           neutral repo name, never the brand name
  apps/
    resident/      Expo app: residents and families (Android, iOS)
    field/         Expo app: ambulance crew, phlebotomists, companions,
riders
    console/       Next.js on Vercel: admin console + partner portal
    web/           Next.js on Vercel: landing site and community sign-up
    api/           Hono on Vercel Functions (bom1): REST, webhooks, Workflows
  packages/
    brand/         Name, domains, contacts, logos: the only place the brand
lives
    contracts/     Zod schemas and shared types; source of the OpenAPI spec
    api-client/    Typed client generated from OpenAPI, used by all apps
    db/            Drizzle schema, migrations, PostGIS helpers, RLS policies
    tokens/        Design tokens: Aurora at launch, structured so more themes
are just new files
    ui-native/     Shared React Native components
    ui-web/        Shared web components (shadcn-based)
    config/        ESLint, TypeScript and Tailwind presets
  supabase/        Supabase config, migrations and seed data for local
development
  design/          Reference HTML files; CareNest-Aurora.html is the one to
build
```

Architecture rules:

- The API is one Hono app split into route modules per domain: `auth`, `households`, `health-records`, `consults`, `calls`, `pharmacy`, `labs`, `companion`, `sos`, `dispatch`, `tracking`, `partners`, `payments`, `notifications`, `analytics`, `audit`. Modules talk through service functions and domain events, never each other's tables.
- Every function is stateless. Anything that must outlive a request lives in Postgres, Upstash Redis or a Workflow run, never in memory.
- Real-time traffic (SOS state, ambulance positions, live queues, console alarms) goes through Supabase Realtime channels per incident, per household and per community desk. Clients subscribe directly; the API publishes.
- Timers that matter (SOS escalation, order SLAs, appointment reminders) are Vercel Workflows with durable `sleep`, never `setTimeout`. Pin every run to `bom1`.
- Set `regions: ["bom1"]` for every Vercel project and keep Supabase in Mumbai, so health data is processed and stored in India. Keep health data out of logs and Workflow step inputs where possible; pass record ids instead.
- Both mobile apps work offline for reading the health record and queue writes; SOS has its own fallback path (next section).
- Every write to medical data, every record share and every SOS action lands in an append-only `audit_events` table.



system architecture · apps, API, data, providers

Every app writes through the API on Vercel; apps subscribe to Supabase Realtime channels directly for live updates, and call media flows between devices and LiveKit using tokens the API issues.

SOS and emergency dispatch

SOS is the feature that must never fail silently: an alert has to reach a human within seconds, even on a weak network, and every step is recorded. Build it as a server-side state machine in the `sos` and `dispatch` modules, with these states: `countdown → raised → dispatching → accepted → en_route → on_scene → transporting → handed_over → closed` (plus `cancelled` and `escalated_to_108`).

Flow:

1. **Trigger.** The resident taps SOS on the home screen or a lock-screen widget. A 5-second cancellable countdown starts, with a "Send now" button. Capture the best available location at once (last known, then a fresh high-accuracy fix).

2. **Raise.** Send `POST /sos` with location, household and an idempotency key. If the request fails or takes longer than 4 seconds, retry with backoff and in parallel offer two fallbacks on screen: "Call community desk" (the desk's phone number, through the phone dialer) and "Call 108". Never block the user on the network.
3. **Notify at once, in parallel:** the community desk (console alarm with sound, push and a WhatsApp alert to the desk's number), family contacts (push and a WhatsApp alert, with SMS if WhatsApp is not delivered within 60 seconds, and no medical details), and gate security if the household is in a community.
4. **Dispatch.** Query free ambulances by distance and road ETA (Upstash Redis GEO for the shortlist, Google Routes for ETA) and offer the incident to the nearest 3 at the same time via critical push and an in-app ringing screen. The first to accept wins through an atomic Redis lock; the others are released.
5. **Escalate on timeout.** If nobody accepts in 30 seconds, widen to the next ring of ambulances and alert the desk to dispatch manually. If nobody accepts in 90 seconds, prompt the resident and desk to call 108 and mark `escalated_to_108`. Make both thresholds configurable. Run the whole dispatch and escalation sequence as one Vercel Workflow per incident, using durable sleeps and a hook that resumes when a crew accepts.
6. **Track.** The field app streams the crew's location every 3 to 5 seconds to the API during an active job, which updates Redis and broadcasts on the incident's Supabase Realtime channel (Android foreground service, iOS background location). The resident app and console show the ambulance moving, the route and an ETA refreshed every 30 seconds.
7. **Hospital handover.** As soon as a crew accepts, choose the receiving network hospital (nearest with capacity, or the resident's preferred one) and send it a secure, time-limited handover link: blood group, allergies, conditions, current medicines, last vitals, emergency contacts and insurance. Share only what the resident has consented to; with consent off, send the blood group only. The ER confirms receipt in the partner portal.
8. **Close.** The crew marks on-scene, transporting and handed-over; the desk closes the incident. Family get a final update.

Hard requirements:

- Every state change is timestamped and kept in `sos_events`; the console shows the full timeline.

- Calls to the crew from the SOS screen use in-app voice. If that cannot connect, offer a one-tap phone call to the crew's company-issued work number, never a personal number.
- A "test SOS" mode for drills that runs the full flow but is clearly labelled and never dispatches a real ambulance.
- Load-test dispatch, and write end-to-end tests that run the whole flow with simulated crews.

In-app calling

All calls run on LiveKit with a native incoming-call screen; when in-app calling cannot connect, the fallback is a WhatsApp message asking the other side to open the app, plus a normal phone call for staff on company-issued numbers. Build one `calls` module that every service uses.

Call types:

Call	Between	Media	Starts from
Video consult	Resident and doctor	Video and audio	Appointment time; either side joins from a waiting room
Audio consult	Resident and doctor	Audio	Same as video
Crew call	Resident or family and ambulance crew	Audio	SOS screen
Service call	Resident and phlebotomist, companion or rider	Audio	Active booking or order
Desk call	Community desk and resident	Audio	Console

Rules:

- The API issues short-lived LiveKit tokens only to the two parties of a booking or incident; room names are opaque and expire.
- Incoming calls ring through CallKit (iOS) and ConnectionService (Android) via VoIP push, so they work with the app closed. On iOS every VoIP push must report a call to CallKit at once, or Apple stops delivering them.

- Doctors join consults from the partner portal in a desktop browser (LiveKit web SDK), with the resident's shared records and an e-prescription form beside the video.
- Consults show a pre-call device check, reconnect automatically after network drops, and drop to audio-only when bandwidth is low.
- If an in-app call fails twice or the other side is offline, send them a WhatsApp message to open the app. For crew, desk and partner staff, also offer a one-tap phone call to their company-issued work number. Never show residents' personal numbers to staff.
- Do not record calls by default. If recording is enabled later, it needs explicit consent from both sides and storage in India.
- Log call metadata (who, when, duration, quality) for support. Never log media.

WhatsApp messaging

WhatsApp is the main channel for anything sent outside the app; there are no masked phone calls. Use Meta's WhatsApp Cloud API directly from the API on Vercel, through one `messaging` module with SMS fallback built in.

- **What goes on WhatsApp:** SOS alerts to family and the community desk, appointment confirmations and reminders, "doctor is waiting" nudges, order and lab-collection status, report-ready alerts, companion trip updates, OTP, and app invites for households not yet on the app.
- **Templates:** keep a versioned template catalogue in code (name, category, language, variables) for English, Telugu and Hindi. Use the utility category for service updates and the authentication category for OTP; keep marketing templates separate and only for users who opted in to them.
- **Never send health details.** Messages say what happened and link into the app ("Your lab report is ready. Open {{brand}} to view it."); no conditions, values, medicines or diagnoses.
- **Opt-in:** record consent for WhatsApp at sign-up with the date, and honour "STOP" replies at once.
- **Webhooks:** receive delivery and read receipts and user replies at `/webhooks/whatsapp`; verify Meta's signature; store status in `message_log`. Simple button replies ("Confirm", "Reschedule") act on bookings.
- **Fallback:** if a critical message (SOS, OTP) is not delivered within 60 seconds, send it by SMS through MSG91.

- **Cost control:** Meta charges per delivered template message by category since July 2025, so batch reminders sensibly and track spend per category in the console.

Features by surface

Match the screens, copy and interactions in the `/design` files; where the mockup uses sample data, build the real data flow behind it.

Resident app (Android, iOS)

- **Onboarding:** phone OTP, language choice (English, Telugu, Hindi to start; all strings in i18n files), household setup (join a community by code or address, or stand alone), add family members, consent screen for record sharing.
- **Home:** greeting, health snapshot (latest BP, pulse, SpO2, sugar with status colours), SOS card, four service tiles, upcoming items, medicine dose check-off.
- **Health:** six vitals with 7-reading trends and target bands, a log-reading sheet with validation, BMI, conditions with status, allergies with severity, medicines with 30-day adherence, connected devices (Bluetooth BP monitor and glucometer later; manual and Health Connect / HealthKit import first), vaccinations, insurance e-card.
- **Consult:** video, audio or clinic; symptom box that suggests a specialty; specialty chips; doctors with fees and slots; choose which records to share; booking, payment, waiting room, call, e-prescription in Records.
- **Medicines:** search, upload a prescription photo or use an e-prescription, quantity steppers, choose a partner pharmacy, allergy check that blocks conflicting drugs, live order tracking, refill reminders.
- **Lab tests:** packages, collection slot, collector details with in-app call, sample status, reports with flagged values, share with a doctor, book a follow-up.
- **Visit companion:** appointment details, trip timeline, transport option, help needed, assigned companion, live trip updates for family.
- **SOS:** countdown, live tracking, crew call, timeline, what was shared with the ER, cancel.
- **Bookings, Records, Profile, Notifications** as in the mockup, plus family accounts so an adult child can manage a parent's care.

Field app (Android, iOS)

- Role-based home for crew, phlebotomist, companion or rider; availability toggle (on duty, off duty).

- Incoming job screen with ring and accept or decline; navigation hand-off to Google Maps; status buttons for each step; in-app call to the resident, with a WhatsApp nudge if they don't answer.
- For crew: patient summary (allergies, conditions, medicines) visible only during an active incident, with a first-aid notes field that is added to the ER handover.
- Background location streaming only while on an active job, with a visible indicator.

Admin console (web)

- Command center: live emergency map, active incident panel with handover record, reassign, call crew, mark handed over, test SOS; vitals alerts; service queue with next-step actions; fleet status.
- Usage heatmap: active users by locality and hour, 7, 30 or 90 days, time-of-day slider with playback, zoom, layers (member communities, network hospitals, ambulance standby points), locality panel with actions (standby point, health camp).
- Resident heatmap: city view by community (everyone, seniors 60+, high risk, living alone) and a site plan per community (villas by lane, flats by tower and floor) coloured by health risk, seniors or vitals due; home panel with home visit and consult booking; needs-a-check-in list.
- Residents directory and profiles, consultations, pharmacy orders, lab collections, companion trips, ambulance fleet, partner network, community onboarding.
- Roles: super admin, operations, community desk (sees only their community and no medical detail beyond what consent allows), support.

Partner portal (web, same console app, role-based)

- Doctors: schedule and slots, consult queue, video call with shared records, e-prescription compliant with India's Telemedicine Practice Guidelines (registration number, format).
- Pharmacies: accept orders, verify prescriptions, mark packed and dispatched.
- Labs: assign collectors, upload reports (PDF plus structured values).
- Hospitals: incoming SOS handovers with arrival ETA, confirm receipt.

Landing site (web)

- Sections from the mockup: hero, who it's for, five services, SOS story, why us, how it works (villas and apartments), plans, FAQ.

- Two-step sign-up form for communities and organisations, saved as a lead in the console with status (new, contacted, walkthrough booked, live) and an email to the sales inbox.
- Broaden the copy so families, senior-living homes and employers can sign up too, not only gated communities.

Data model outline

Start from these core tables and refine them as you build; every table has `id` (UUID), `created_at`, `updated_at`, and soft delete where records are medical.

Area	Tables
Identity	`users`, `user_roles`, `staff_accounts`, `sessions`, `otp_challenges`
Customers	`organisations` (community, senior home, employer), `sites` (gated community or building, with PostGIS boundary), `homes` (villa or flat, with tower, floor, number, point location), `households`, `household_members`, `emergency_contacts`
Health record	`health_profiles` (blood group, height), `vital_readings` (type, value, unit, source, taken_at), `conditions`, `allergies`, `medications`, `medication_doses`, `vaccinations`, `insurance_policies`, `devices`
Consent	`consents` (purpose, scope, granted_at, revoked_at), `record_shares` (who saw what, when, why)
Consults	`doctors`, `specialties`, `doctor_slots`, `appointments`, `consult_notes`, `prescriptions`, `prescription_items`
Calls	`calls` (type, parties, room, started_at, ended_at, quality)
Pharmacy	`pharmacies`, `medicine_catalog`, `orders`, `order_items`, `order_events`

Area	Tables
Labs	`labs`, `test_packages`, `lab_bookings`, `lab_reports`, `lab_results`
Companion	`companion_requests`, `companion_trips`, `trip_events`
SOS	`ambulances`, `crews`, `crew_shifts`, `sos_incidents`, `sos_events`, `dispatch_offers`, `hospital_handovers`, `standby_points`
Partners	`hospitals` (with capacity flags), `partner_users`
Operations	`home_visits`, `health_camps`, `leads` (landing sign-ups), `notifications`, message_log (WhatsApp and SMS, with delivery status), messaging_optins, `payments`, `audit_events`
Analytics	`app_sessions` (coarse locality and hour only, no raw GPS), daily rollups for both heatmaps

Resident heatmap counts and risk tiers come from the health record and consent tables at query time or from nightly rollups; never store a separate copy of medical facts for analytics.

Security, privacy and India compliance

Treat every health field as sensitive personal data: keep it in India, encrypt it, share it only with recorded consent, and log every access. Build for these requirements, and flag anything that needs a lawyer's review rather than deciding it in code.

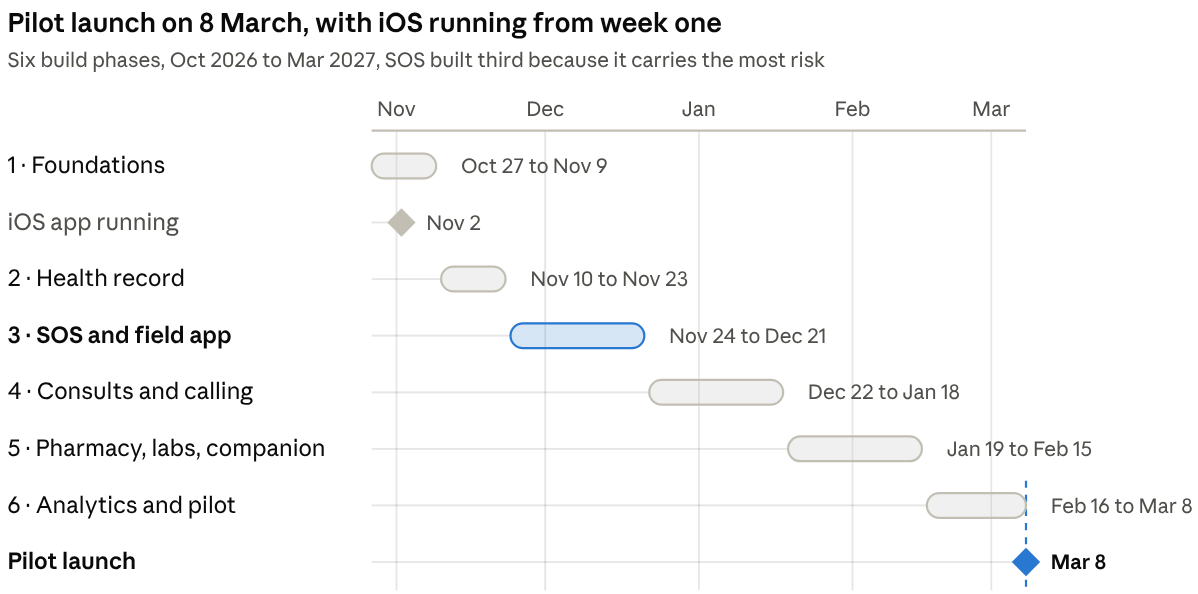
- **Digital Personal Data Protection Act, 2023 (DPDP):** purpose-specific consent with plain-language notices, easy withdrawal, access and erasure requests from the Profile screen, breach-response runbook, data-retention rules per table.
- **Telemedicine Practice Guidelines (2020):** doctor identity and registration number shown before a consult, patient consent captured, e-prescription format, no prescribing of restricted drug lists over teleconsultation.
- **Medicines:** only licensed partner pharmacies dispense; prescription-only drugs require a verified prescription; keep pharmacy licence numbers on file.

- **SMS:** register sender IDs and templates on the DLT platform before launch; OTP and alerts only through approved templates.
    - **WhatsApp:** Meta business verification, a recorded opt-in before any message, approved templates for everything sent outside the 24-hour window, and no health details in any message.
- **Google Play:** complete the health apps declaration, data safety form and background-location declaration (used only during active field jobs and SOS); use a foreground service type for location.
- **Apple App Store:** request the Critical Alerts entitlement for SOS; follow the VoIP push rules (every push reports a CallKit call); background location justification; privacy nutrition labels.
- **ABDM:** design the health record so ABHA linking and FHIR export can be added later; not required for launch.

Engineering controls: TLS everywhere, encryption at rest in Supabase, field-level encryption for notes and report files, least-privilege roles, 2FA for staff, rate limits on OTP and SOS endpoints, secrets only in Vercel and Supabase encrypted environment variables, dependency and container scanning in CI, and a quarterly restore test of backups.

Delivery plan

Build in six phases starting 27 Oct 2026, with Android and iOS running from the same code from week one and a pilot in one community by early March 2027. Dates assume one developer working with Claude Code full time; adjust them to your team. Each phase ends with a demo on a real Android phone and a real iPhone.



delivery roadmap · 6 phases, 27 Oct 2026 to 8 Mar 2027

1. **Foundations (27 Oct – 9 Nov).** Monorepo, CI, Vercel projects for web, console and api pinned to bom1, Supabase project in Mumbai, auth with phone OTP, Aurora design tokens, app shells for resident and field apps running on Android and iOS by 2 Nov, console and web shells. Outside the code: enrol in the Apple Developer Program and Google Play Console, request Critical Alerts, start DLT registration, open LiveKit, MSG91, Razorpay and Google Maps accounts, and set up the WhatsApp Business Platform (Meta business verification, a dedicated number, first templates submitted for approval).
2. **Households and health record (10 – 23 Nov).** Organisations, sites, homes, households, family accounts, consent, the full Health tab and the home snapshot, residents directory in the console.
3. **SOS and field app (24 Nov – 21 Dec).** The full SOS state machine, dispatch with escalation, field app jobs and location streaming, live tracking, hospital handover, console command center, test SOS. This phase comes early because it is the riskiest and the core promise.
4. **Consults and calling (22 Dec – 18 Jan).** Doctor onboarding, slots, booking and payment, LiveKit calls with CallKit and ConnectionService, doctor portal, e-prescriptions, WhatsApp fallback when a call can't connect.
5. **Pharmacy, labs and companion (19 Jan – 15 Feb).** Ordering with allergy checks, pharmacy portal, lab bookings and reports, companion trips with live updates, refill and appointment reminders.

6. **Analytics, landing and pilot (16 Feb – 8 Mar).** Usage and resident heatmaps, landing site with lead capture, load tests, security review, store submissions, pilot launch in one community.

Exit criteria for every phase: all screens match the design files, unit and end-to-end tests pass, Sentry shows no unhandled errors in a day of internal use, and the README explains how to run that part locally.

How you work (rules for Claude)

Work one phase at a time, in small reviewed steps, and stop to ask whenever a decision is expensive to undo.

1. **Start every session** by reading this file and the latest entry in `docs/progress.md`, then state the next task in one line before writing code.
2. **Plan before building** any module: write a short plan (endpoints, tables, screens, tests) to `docs/plans/<module>.md` and wait for approval on phases 3 and 4 (SOS and calling).
3. **Never invent provider APIs.** Read the current docs for Vercel, Supabase, Expo, LiveKit, CallKeep, WhatsApp Cloud API, MSG91, Razorpay and Google Maps before integrating; pin versions in `package.json`.
4. **Secrets** only in `.env` files that are git-ignored, with a committed `.env.example`. Use sandbox or test keys until launch.
5. **Type safety:** strict TypeScript, Zod validation on every API input, generated API client in the apps, no `any` without a comment explaining why.
6. **Tests with every feature:** unit tests for domain logic, API integration tests against a local Supabase stack, Maestro flows for each mobile journey, Playwright for the console and landing. SOS and calling need tests for the failure paths too.
7. **Accessibility and India-first UX:** large tap targets (44 px minimum), screen-reader labels, works on low-end Android phones and 3G, all text translatable, rupee formatting with Indian digit grouping.
8. **Commits** are small and use conventional messages; open a pull request per feature with a summary and screenshots from both Android and iOS.
9. **Update `docs/progress.md`** at the end of each session: what was done, what is next, open questions.
10. **Ask before:** adding a paid service, changing the stack above, storing new kinds of health data, or anything that touches consent or SOS escalation rules.

Your first task: scaffold Phase 1 (Foundations). Create the monorepo, the five app shells and shared packages, wire design tokens from `/design` for the Aurora theme (from CareNest-Aurora.html), get the resident app running on an Android emulator and an iOS simulator, and write `docs/progress.md`.

Sources

Vercel and Supabase facts above were checked on 26 Oct 2026:

- Vercel regions: `bom1` is Mumbai (ap-south-1); functions default to `iad1` unless changed.
- Vercel Functions limits: maximum duration 300 s on Hobby, 800 s on Pro.
- WebSockets on Vercel Functions: supported, but connections close at the function's maximum duration and state must live outside the function.
- Vercel Workflows: durable sleeps, hooks and retries, with runs pinned to a chosen region.
- Vercel Daemons: background containers, still in public beta and not usable from a subfolder app in a monorepo, so not used here.
- Supabase regions: Mumbai (ap-south-1) is available.
- WhatsApp Business Platform pricing: per-message pricing by category since 1 July 2025; template messages outside the 24-hour customer service window; opt-in required.